

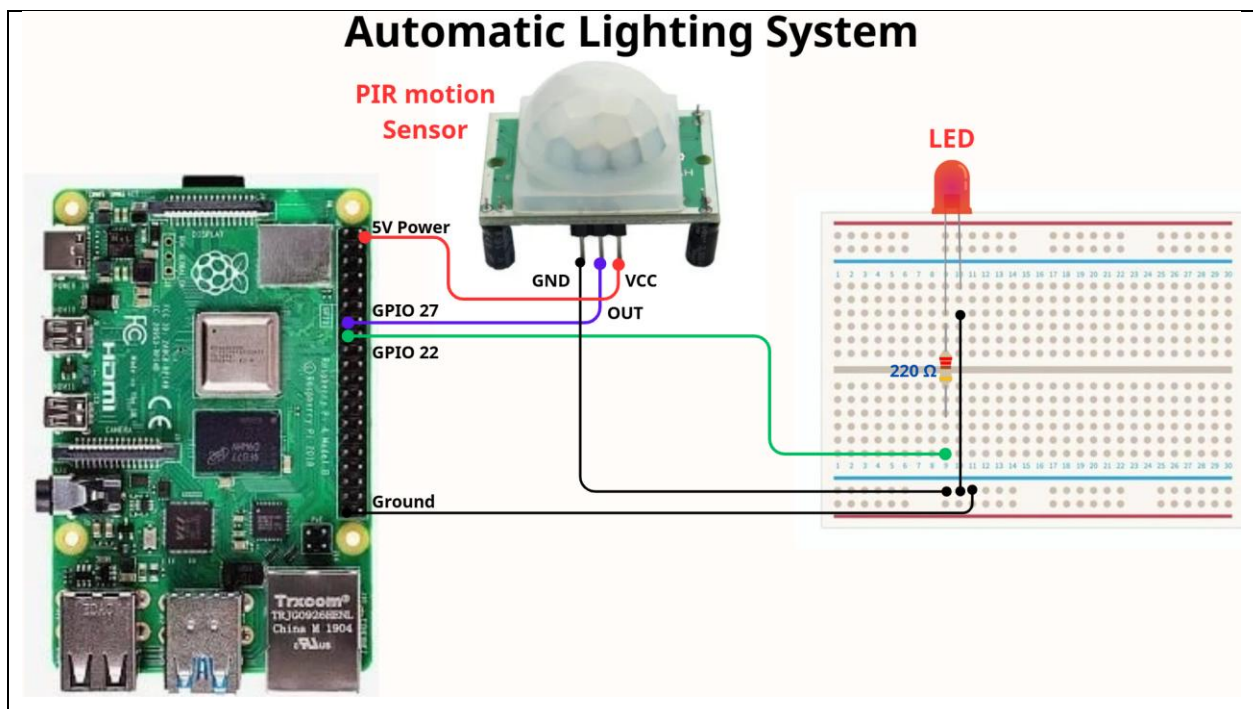
Project: IoT Smart Home for Monitoring and Automation

Author: Nguyen Cong Dung (ITITI21184)

1. Development Environment & Hardware Setup

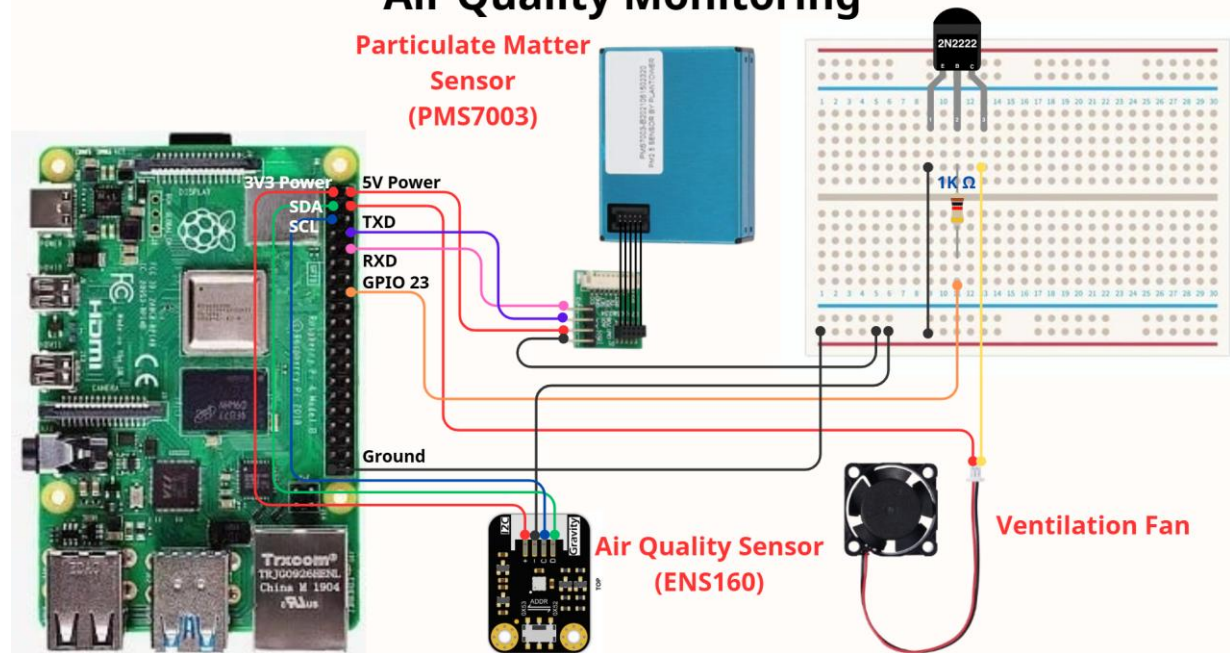
The system utilizes a split edge-computing architecture.

- **Host Computer:** Windows OS (used for coding, file transfers, and VNC remote desktop control).
- **Edge Platform:** Raspberry Pi 4 (Running Linux-based Raspberry Pi OS).
 - **IP Address:** 10.103.22.246.
- **Hardware Interface Configuration:**
 - Connect the Raspberry Pi 4 to a local WLAN network.
 - Open terminal and run `sudo raspi-config` to enable the **I2C** and **UART** interfaces.
- **Hardware Diagram**

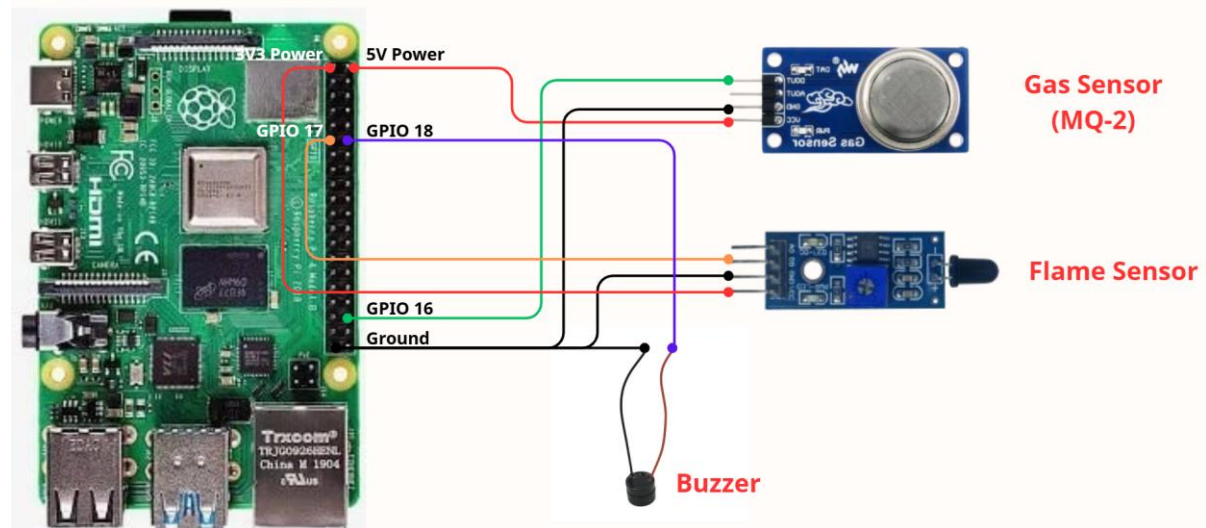


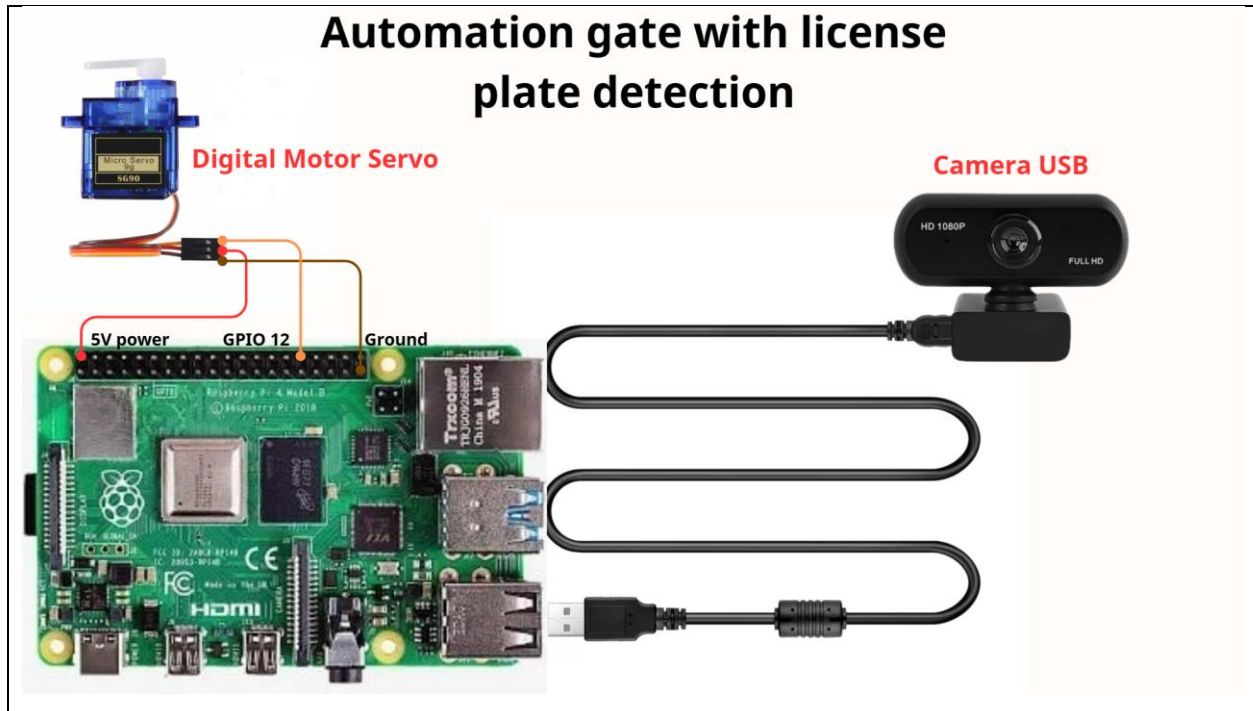
Air Quality Monitoring

Particulate Matter
Sensor
(PMS7003)



Fire Alert System





2. Software Dependencies & Installation

All software components are developed using **Python 3.12**. Run the following commands on your Raspberry Pi terminal to install the required libraries:

For the Smart Home & Fire Alert Node (FireDetect.py)

```
pip install paho-mqtt adafruit-circuitpython-ens160 gpiozero pyserial
```

- paho-mqtt: Connects and uploads sensor telemetry data to ThingsBoard via MQTT.
- pyserial: Parses raw 32-byte streams from the PMS7003 dust sensor.
- adafruit-circuitpython-ens160: Drives the I2C-based ENS160 air quality sensor.
- gpiozero: High-level library to read digital inputs (MQ-2 Gas, Flame, PIR) and control outputs (Buzzer, Fan, LED).

For the License Plate Recognition Gate (main_anpr.py)

```
pip install ultralytics easyocr opencv-python torch torchvision flask python-bidi pillow
```

- ultralytics: Loads and executes the **YOLOv8n** plate localization model.
- easyocr: Extracts text strings from localized plate images.
- opencv-python (v4.9.0): Handles video stream acquisition, frame resizing, and color space transformations.
- flask: Hosts the local web interface for remote live camera streaming.

3. Deployment & Execution Steps

Step 1: Model Deployment

1. Download your optimized weights file **best.pt** (obtained via transfer learning on the 4,437 Vietnamese license plate dataset).
2. Place best.pt and Database.txt directly into the same working directory as your license plate recognition script.

Step 2: Configure Cloud Connectivity

Ensure your ThingsBoard platform device access tokens match the following parameter inside both python scripts:

Python

```
ACCESS_TOKEN = "klk3AZg0vQdXkUDCyYdj"
```

```
THINGSBOARD_HOST = "demo.thingsboard.io"
```

Step 3: Run the Systems

Open two separate terminals on your Raspberry Pi (via VNC) and execute:

- **To start Environmental Safety Monitoring:**

```
cd ~/Thesis/Plate
```

```
source .venv/bin/activate
```

```
python FireDetect.py
```

- **To start the Automated Gate Control:**

```
cd ~/Thesis/Plate
```

```
source .venv/bin/activate
```

```
export PYTHONPATH=/home/pi/Thesis/Plate/.venv/lib/python3.12/site-packages
```

```
python main_anpr.py
```

```
ngrok http 5000
```

*Open a web browser on your host computer and navigate to

`[http://10.103.22.246:5000](http://10.103.22.246:5000)` to monitor the camera feed.*

4. Model Re-Training Summary (For Reference)

If you need to reproduce or re-train the AI model:

- **Architecture:** YOLOv8n (Nano profile chosen for edge hardware speed).
- **Dataset:** 4,437 images of Vietnamese traffic scenarios (cars, motorcycles, extreme lighting, glare) labeled in YOLO .txt coordinate format.
- **Input Resolution:** 640 $\times$ 640 pixels.
- **Results:** Attained **92.1% Precision**, **94.5% Recall**, and an **mAP50 score of 0.967**, yielding an inference latency of 150–200 ms on the edge platform.